Dynamic Proxies

Ghost Methods (57) are usually icing on the cake, but some objects actually rely almost exclusively on them. These objects are often wrappers for something else—maybe another object, a web service, or code written in a different language. They collect method calls through method_missing and forward them to the wrapped object. Let's look at a complex real-life example.

The Ghee Example

You probably know GitHub,[1] the wildly popular social coding service. A number of libraries give you easy access to GitHub's HTTP APIs, including a Ruby gem called Ghee. Here is how you use Ghee to access a user's "gist"—a snippet of code that can be published on GitHub:

methods/ghee_example.rb

```
require "ghee"

gh = Ghee.basic_auth("usr", "pwd")  # Your GitHub username and password
all_gists = gh.users("nusco").gists
a_gist = all_gists[20]

a_gist.url            # => "https://api.github.com/gists/535077"
a_gist.description    # => "Spell: Dynamic Proxy"

a_gist.star
```

The code above connects to GitHub, looks up a specific user ("nusco"), and accesses that user's list of gists. Then it selects one specific gist and reads that gist's url and description. Finally, it "stars" the gist, to be notified of any future changes.

The GitHub APIs expose tens of types of objects besides gists, and Ghee has to support all of those objects. However, Ghee's source code is surprisingly concise, thanks to a smart use of *Ghost Methods (57)*. Most of the magic happens in the Ghee::ResourceProxy class:

gems/ghee-0.9.8/lib/ghee/resource_proxy.rb

```
class Ghee
  class ResourceProxy
    # ...

    def method_missing(message, *args, &block)
      subject.send(message, *args, &block)
    end
```

1. http://www.github.com